

✓ PYTHON DATA STRUCTURES

Python has 4 built-in data structures:

1. List -> [] (Mutable)
2. Tuple -> () (Immutable)
3. Set -> {} (Unique values only)
4. Dictionary -> {} (Key : Value pairs)

```
# a = [1,"abc",12321.3,True]

fruits = ["Apple", "Banana", "Mango","carrot", "potato"]
# 0      1      2      3
# -4     -3     -2     -1

len(fruits)

fruits[0]

fruits[3] = "veges"

type(fruits)

fruits.append("dragon fruit") # list_name.append()

fruits.remove("Mango") # list_name.remove()

len(fruits)

fruits[1:10]

['Banana', 'veges', 'potato', 'dragon fruit']
```

```
# =====
# 1. LIST
# =====

# A List is an ordered collection of items.
# Lists are MUTABLE, meaning we can add, remove,
# or modify elements after creating the list.

fruits = ["Apple", "Banana", "Mango"]

print(fruits)

# Accessing elements using index
# Index always starts from 0

print("\nFirst Fruit:")
print(fruits[0])

print("Second Fruit:")
print(fruits[1])

# Negative indexing starts from the end

print("\nLast Fruit:")
print(fruits[-1])

# Modifying an element

fruits[1] = "Orange"

print("\nAfter Modifying:")
print(fruits)

# Adding an element

fruits.append("Grapes")
```

```

print("\nAfter Append:")
print(fruits)

# Removing an element

fruits.remove("Apple")

print("\nAfter Remove:")
print(fruits)

# Length of the list

print("\nLength of List:")
print(len(fruits))

```

```

days = ("Monday", "Tuesday", "Wednesday")

type(days)

# accessing the element

days[2]

a = ("book", "book")
a

```

```

('book', 'book')

```

```

# =====
# 2. TUPLE
# =====

# A Tuple is also an ordered collection.
#
# Tuples are IMMUTABLE.
#
# Immutable means:
# Once a tuple is created,
# its values CANNOT be changed.

days = ("Monday", "Tuesday", "Wednesday")

print("\n\nTuple:")
print(days)

# Accessing elements

print("\nFirst Day:")
print(days[0])

print("Last Day:")
print(days[-1])

print("\nLength of Tuple:")
print(len(days))

# Uncomment the code below to see the error

# days[1] = "Friday"

# Error:
# TypeError:
# 'tuple' object does not support item assignment

# This happens because tuples are immutable.

```

```

s = {"b", "a", "z"}
s

```

```
{ 'a', 'b', 'z' }
```

```
numbers_set = {"@",1, 3,2, 4, 5, 5, 5 , "a","z","b"}
```

```
numbers_set
```

```
{1, 2, 3, 4, 5, '@', 'a', 'b', 'z'}
```

```
numbers_set = {1, 3, 4,2, 4, 5, 5, 5 , "a","z","b"}
```

```
numbers_list = [ 1,2,4,4,2,5]
```

```
numbers_tuple = (1,2,4,4,2,5)
```

```
print(numbers_list)
```

```
print(numbers_tuple)
```

```
print(numbers_set)
```

```
[1, 2, 4, 4, 2, 5]
```

```
(1, 2, 4, 4, 2, 5)
```

```
{ 'a', 1, 2, 3, 4, 5, 'z', 'b' }
```

```
numbers = {1,2,3,4,4,5,4}
```

```
type(numbers)
```

```
numbers.add(5)
```

```
print(numbers)
```

```
len(numbers)
```

```
{1, 2, 3, 4, 5}
```

```
5
```

```
# =====  
# 3. SET  
# =====
```

```
# A Set is an unordered collection.  
#  
# Sets DO NOT allow duplicate values.  
#  
# Sets automatically remove duplicates.
```

```
set = {}
```

```
print("\n\nSet:")
```

```
print(numbers)
```

```
# Output will contain only unique values.
```

```
# Adding an element
```

```
numbers.add(10)
```

```
print("\n\nAfter Adding:")
```

```
print(numbers)
```

```
# Removing an element
```

```
numbers.remove(2)
```

```
print("\n\nAfter Removing:")
```

```
print(numbers)
```

```
print("\n\nLength of Set:")
```

```
print(len(numbers))
```

```
# NOTE:
```

```
# Sets DO NOT support indexing.
```

```
#
```

```
# Example:
```

```
#
# numbers[0]
#
# This gives an error because sets are unordered.
```

```
Set:
{1, 2, 3, 4, 5}

After Adding:
{1, 2, 3, 4, 5, 10}

After Removing:
{1, 3, 4, 5, 10}

Length of Set:
5
```

```
student = { "name": "John", "age": 21, "course": "Data Science"}

student["name"] # accessing the value using key
student['age'] = 22 # updating the record

student['city'] = 'Trichy' # to add new key & value

student.pop("age") # to remove an data

# type(student)

# help(list)
print(student)

# student.keys()
# student.values()
student.items()
```

[Show hidden output](#)

```
# =====
# 4. DICTIONARY
# =====

# A Dictionary stores data in
# KEY : VALUE pairs.
#
# Keys should be unique.
#
# Dictionaries are MUTABLE.
name = "John"
student = { "name": "John",

            "age": 21,
            "course": "Data Science"
          }

print("\n\nDictionary:")
print(student)

# Accessing values using keys

print("\nStudent Name:")
print(student["name"])

print("Student Age:")
print(student["age"])

# Modifying a value

student["age"] = 22

print("\nAfter Updating Age:")
print(student)

# Adding a new key-value pair

student["city"] = "Chennai"
```

```
print("\nAfter Adding City:")
print(student)

# Removing a key-value pair

student.pop("course")

print("\nAfter Removing Course:")
print(student)

print("\nNumber of Key-Value Pairs:")
print(len(student))
```

```
# =====
# SUMMARY
# =====

# LIST
# ----
# Symbol      : []
# Ordered     : Yes
# Mutable     : Yes
# Duplicates  : Allowed
# Indexing    : Yes

# TUPLE
# -----
# Symbol      : ()
# Ordered     : Yes
# Mutable     : No (Immutable)
# Duplicates  : Allowed
# Indexing    : Yes

# SET
# ---
# Symbol      : {}
# Ordered     : No
# Mutable     : Yes
# Duplicates  : NOT Allowed
# Indexing    : No

# DICTIONARY
# -----
# Symbol      : {}
# Stores      : Key : Value pairs
# Keys        : Unique
# Mutable     : Yes
# Indexing    : No (Access using keys)

# =====
# COMPARISON TABLE
# =====

# +-----+-----+-----+-----+-----+
# | Structure | Symbol | Mutable? | Duplicates | Indexing? |
# +-----+-----+-----+-----+-----+
# | List      | []    | Yes      | Yes        | Yes       |
# | Tuple     | ()    | No       | Yes        | Yes       |
# | Set       | {}    | Yes      | No         | No        |
# | Dictionary | {}    | Yes      | Keys: No   | By Keys   |
# +-----+-----+-----+-----+-----+
```

